Vue Components

Vue使用规范（根据以下7点规范，修改本项目）：

1. 使用多单词组件名

- 除根组件 App 外，用户组件的名称应始终由多个单词组成。这样可以避免与现有及未来的 HTML 元素发生冲突，因为所有 HTML 元素都是单个单词的。

Bad

```template
<!-- in pre-compiled templates -->
<Item />

<!-- in in-DOM templates -->
<item></item>
```

Good

```template
<!-- in pre-compiled templates -->
<TodoItem />

<!-- in in-DOM templates -->
<todo-item></todo-item>
```

2. 使用详细的 prop 定义

- 在提交的代码中，prop 的定义应尽可能详细，至少需要明确指定其类型。

Bad

```js
// This is only OK when prototyping
const props = defineProps(['status'])
```

Good

```js
const props = defineProps({
  status: String
})
```

```js
// Even better!

const props = defineProps({
  status: {
    type: String,
    required: true,

    validator: (value) => {
      return ['syncing', 'synced', 'version-conflict', 'error'].include
        value
      )
    }
  }
})
```

Vue Components

3. 使用组件作用域的样式

- 在应用中，顶层 App 组件和布局组件的样式可以是全局的，但其余所有组件的样式都应限定在组件作用域内。

Bad

```template
<template>
  <button class="btn btn-close">×</button>
</template>

<style>
.btn-close {
  background-color: red;
}
</style>
```

Good

```
<template>
  <button class="button button-close">×</button>
</template>

<!-- Using the `scoped` attribute -->
<style scoped>
.button {
  border: none;
  border-radius: 2px;
}

.button-close {
  background-color: red;
}
</style>
```

4. 组件文件规范

- 每个组件都应独立存放在专属文件中。

Bad

```js
app.component('TodoList', {
  // ...
})

app.component('TodoItem', {
  // ...
})
```

Good

```
components/
|- TodoList.js
|- TodoItem.js
```

```
components/
|- TodoList.vue
|- TodoItem.vue
```

5. 单文件组件的文件名格式

- 单文件组件的文件名应统一使用 PascalCase（大驼峰式）或 kebab-case（短横线连接式）。

Bad

```
components/
|- mycomponent.vue
```

```
components/
|- myComponent.vue
```

Good

```
components/
|- MyComponent.vue
```

```
components/
|- my-component.vue
```

6. 紧密耦合组件的命名规范

- 与父组件存在紧密耦合关系的子组件，其名称应以父组件名作为前缀。

Bad

```
components/
|- TodoList.vue
|- TodoItem.vue
|- TodoButton.vue
```

```
components/
|- SearchSidebar.vue
|- NavigationForSearchSidebar.vue
```

Good

```
components/
|- TodoList.vue
|- TodoListItem.vue
|- TodoListItemButton.vue
```

```
components/
|- SearchSidebar.vue
|- SearchSidebarNavigation.vue
```

Vue Components

7. 指令缩写使用规范

- 指令缩写（如 : 代替 v-bind:、@ 代替 v-on:、# 代替 v-slot）应在项目中保持统一：要么全部使用缩写，要么全部不使用缩写。

Bad

```
<input
  v-bind:value="newTodoText"
  :placeholder="newTodoInstructions"
>
```

```
<input
  v-on:input="onInput"
  @focus="onFocus"
>
```

```
<template v-slot:header>
  <h1>Here might be a page title</h1>
</template>

<template #footer>
  <p>Here's some contact info</p>
</template>
```

Good

template

```
<input
  :value="newTodoText"
  :placeholder="newTodoInstructions"
>
```

template

```
<input
  v-bind:value="newTodoText"
  v-bind:placeholder="newTodoInstructions"
>
```

template

```
<input
  @input="onInput"
  @focus="onFocus"
>
```

template

```
<input
  v-on:input="onInput"
  v-on:focus="onFocus"
>
```

template

```
<template v-slot:header>
  <h1>Here might be a page title</h1>
</template>

<template v-slot:footer>
  <p>Here's some contact info</p>
</template>
```

浙江大学 管理学院
SCHOOL OF MANAGEMENT
ZHEJIANG UNIVERSITY

03 本次作业



小组作业：用VUE重构小组项目代码

- 使用 npm create vue@latest 作为代码框架
- 所有数据全部保存于json文件。json存放于public文件夹。vue文件、HTML文件、JS文件中不可包含数据
- 利用vue component重构项目。除了root component外至少包括6个components。
- 一个component中，包含所有相关的css、HTML、JS代码。样式使用<style scoped>标签，JS代码使用<script setup>标签。
- 支持使用UI组件库。assets里只能包含第三方库的CSS文件，不能包含自定义的CSS文件。
- 在确保可读性的前提下，所有vue文件中的HTML代码不能超过50行。如果超过，就将部分内容封装成component。
- 项目内容、样式保持不变
- 不能使用var
- 使用composition API

小组作业：用VUE重构项目代码

- 使用composition API，不要使用options API

App.vue - Options API

```vue
<script>
import WordCard from './components/WordCard.vue'

export default {
    components: { WordCard },
    data() {
        return {
            words: …
            correctCount: 0,
            completed: false
        }
    },
    computed: {
        shuffledWords() {
            return this.words.sort(() => .5 - Math.random());
        },
        wordCount() {
            return this.words.length;
        }
    },
    watch: {
        correctCount() {
            this.completed = this.correctCount == this.wordCount;
        }
    },
    methods: {
        incrementCorrectCount() {
            this.correctCount++;
        }
    }
}
</script>
```

App.vue - Composition API

```vue
<script setup>
import { ref, computed, watch } from 'vue'
import WordCard from './components/WordCard.vue'

// Data properties
const correctCount = ref(0);
const correct = ref(false);
const completed = ref(false);
const words = ref([…
]);

// Computed properties
const shuffledWords = computed(() => {
    return words.value.sort(() => .5 - Math.random());
});

const wordCount = computed(() => {
    return words.value.length;
});

// Watchers
watch(correctCount, (newValue, oldValue) => {
    completed.value = correctCount.value == wordCount.value;
});

// Methods
const incrementCorrectCount = () => {
    correctCount.value++;
}
</script>
```

小组作业：用VUE重构项目代码

评分：

- 是否按时?
- 各功能是否能正常运行?
- 是否符合要求?
- 是否使用composition API?